

Path Planning For Moving A Point Object
Amidst Unknown Obstacles In A Plane:
A New Algorithm And A General Theory
For Algorithm Development.¹

A. Sankaranarayanan

M. Vidyasagar

Department of Electrical Engineering
University of Waterloo
Waterloo, Ontario N2L 3G1, Canada

Center for Artificial Intelligence and Robotics
Raj Bhavan Circle, High Grounds
Bangalore - 560 001, India

ABSTRACT

A new algorithm *Alg2* is developed to solve the problem (Lumelsky and Stepanov [6]) of non-heuristically moving a point object (*MA*), between two given points, amidst unknown obstacles in a plane. The existing algorithms to solve this are *Bug1* and *Bug2* (Lumelsky and Stepanov [6]) and *Alg1* (Sankar and Vidyasagar [11]).

The existing algorithms make *MA* follow mostly the contours of the obstacle boundaries. Under the new algorithm *Alg2*, *MA* follows the obstacle boundaries less and travels more along straight line segments towards the target point. This may reduce the path length and relieve to some extent, the sensing system on *MA* from the task of faithfully following an obstacle boundary. The worst case path length, convergence, local cycle creation, etc. of *Alg2* are presented along with examples of its operation.

The algorithm *Alg2* is developed step by step, by adding new procedures to a certain primitive algorithm. This algorithm development brings out the basic requirements of a non-heuristic path planning algorithm, and this knowledge leads to a general algorithm *Gen* of which the algorithms *Bug2*, *Alg1* and *Alg2* are particular cases. This generalization helps understand the basic operating principles of a non-heuristic path planning algorithm.

1. Introduction

A practical robotic application involves moving a robot (mobile or fixed to the floor) amidst obstacles, without colliding with them. This problem of obstacle avoidance has been extensively studied and has generated a vast literature.

The research on path planning in the presence of obstacles can be classified into two large categories. In the first category, popularly known as Piano Movers' problem, an object (generally polyhedral) is to be moved in a scene filled with other polyhedral obstacles. A complete knowledge of the obstacles (i.e., their sizes, positions, orientations, etc.) is assumed. A path is generated through a graph search, in a graph formed out of the connectivity relations of obstacles and free regions in the scene. This one-time, off-line and computationally very intensive procedure is most suitable for repetitive operations in an unchanging environment. Yap [1] reviews various issues and methods to solve problems of first category.

The second category of path planning deals with robot applications involving unknown (or partly known) and changing environments. The sensors (e.g., tactile) fitted on the robot, supply it with information on the obstacles in the immediate neighborhood of the robot. Naturally, this information can only be a local or a partial description of robot's environment. The robot's path is constructed on-line, point by point, based on this local description of the environment.

A path based only on local information may not always exist, because the global requirement of moving a robot between two given points generally requires a global path planning with complete description of the environment (i.e. the first category). This results in one more classification. A path planning scheme of second category, which can not always guarantee a path can be called, heuristical. Most of the schemes of second category, available in the literature, are heuristical. Some of them are discussed next.

Khatib [2] models a repulsive potential field around an obstacle to push the manipulator links away from the obstacle. Maciejewski and Klein [3] propose a similar scheme for a planar redundant manipulator. These schemes can not always guarantee a path, because the manipulator can either get trapped or keep oscillating, between the obstacles, under certain conditions. Chattergy [4], proposes some heuristics to move a mobile robot, fitted with sonar for proximity sensing, in an unknown environment. Several heuristical schemes for mobile robot navigation can be found in the literature.

A path planning scheme of second category, which can always guarantee a path, when there exists one, can be called non-heuristic or provable. Some of the existing non-heuristic schemes are considered next. The Pledge Algorithm [5], generates a path for a point object to escape from a maze whose global description is not known.

The problem studied in this paper was formulated by Lumelsky and Stepanov [6], and can be stated as follows:

Problem P1: *A mobile automaton (MA), which is a point object, has to be continuously moved between two given points S and T, in a plane filled with unknown obstacles of arbitrary shapes. The obstacle boundaries are simple closed curves. At any point on its path, MA knows only its current coordinates and those of target T. MA can store a few points from its path and use them for planning the future path sections. Also, MA can detect and follow an obstacle boundary when it meets one. No other information on the environment is available.*

Two non-heuristic algorithms namely, *Bug1* and *Bug2*, were proposed by Lumelsky and Stepanov [6,7] to solve problem P1. Another non-heuristic algorithm *Alg1* was proposed by Sankar and Vidyasagar [11]. The worst case path length, expressed in terms of perimeters of obstacles met by *MA* and the length *D* of line segment *ST*, is generally indicative of the path lengths produced by an algorithm. The worst case path lengths of the existing algorithms are as follows [6,11]:

$$M_{Bug1} = 1.5 \sum p_i + D, \quad (1)$$

$$M_{Bug2} = \sum p_i n_i / 2 + D, \quad (2)$$

$$M_{Alg1} = 2 \sum p_i + D, \quad (3)$$

¹ This research was supported by the Natural Sciences and Engineering Research Council of Canada under Grant A-1240.

where p_i is the i_{th} obstacle met by MA and n_i is the number of intersections the line segment ST makes with that obstacle boundary.

In Sankar [12], all possible algorithms which solve problem $P1$ are grouped into two classes. Under a Class I algorithm, MA first travels along the entire boundary of every obstacle it meets, before leaving it (e.g. *Bug1*). On the other hand, under a Class II algorithm, MA leaves at least one obstacle before traveling along its entire boundary (e.g. *Alg1* and *Bug2*). It is also shown in [12] that algorithm *Bug1* has the smallest worst case path length possible for any Class I algorithm, and algorithm *Alg1* has the smallest worst case path length possible for any class II algorithm.

These algorithms are based on the Jordan Curve Theorem, and hence can not generally be applied to environments of dimension greater than two. The application of these algorithms to 2-link manipulator obstacle avoidance, taking into account the kinematics of different types of manipulators, is presented in Lumelsky [8,9]. The kinematic constraints arising out of the way in which the links are attached, enable the application of *Bug2* to a 3-link Cartesian Spatial Manipulator path planning (Lumelsky [10]).

This paper presents a new non-heuristic algorithm to solve problem $P1$ stated above. The motivation for the new algorithm is presented after the model definition.

2. Model Definition and Assumptions

These definitions (mostly due to Lumelsky and Stepanov [6]), arising out of problem $P1$, model the scene and MA .

Model M1

A scene (or environment) consists of obstacles and the points Start (S) and Target (T), all lying in a plane. The obstacle boundaries are simple closed curves of finite lengths. An obstacle boundary intersects any straight line only at finite number of points. Though, there is no restriction on the number of obstacles in a scene, any circular disc of finite radius is assumed to intersect only a finite number of obstacles. There is no restriction on the size and shape of an obstacle.

The *Mobile Automaton* (MA) is a point object. At any instant, MA knows, with respect to a reference point, its current coordinates and those of target T . The sensors on MA let it detect and follow an obstacle boundary, when one is met. The MA , after hitting an obstacle can follow the obstacle boundary in two ways. Due to the absence any a priori information on the scene, it is not possible to decide, whether MA should turn right or left to follow the obstacle boundary. Hence, for clarity and without loss of generality, the local direction at the obstacle boundary is assumed to be "left". The point at which MA meets an obstacle is called a hit point H . After moving along the obstacle boundary, MA may leave the obstacle boundary at a leave point L .

The path generated under an algorithm, say *Bug1*, is referred to as P_{Bug1} and its length is denoted as L_{Bug1} . p_i is the perimeter of the i_{th} obstacle met by MA on its way from S to T , and D is the length of the line segment ST . $d(x, y)$ is the Euclidean distance between two points x and y , belonging to the scene. $d_p(x, y)$ denotes the distance traveled by MA along its path (i.e. path length) while moving from a point x to a point y . The path length along the path is determined by integrating the incremental lengths at every step. The total path length $d_p(S, x)$ up to a current position x is stored in a path length counter PC .

A path is said to contain a local cycle if MA visits any point on its path more than once.

An algorithm solves problem $P1$, if it can take MA from S to T or conclude that T is not reachable, within a finite path length generation.

3. Motivation

One of the goals is to design an algorithm so that shorter paths are generated, in general, as compared to the existing algorithms.

The algorithm *Bug1*, though has a lower worst case path length, produces rather long paths for simple scenes (e.g. a scene with only convex obstacles), as MA has to go around the entire perimeter of each obstacle it meets. The algorithms *Alg1* and *Bug2* produce the same path for scenes which do not produce local cycles. When complex scenes produce local cycles, *Alg1* generally produces much shorter paths than *Bug2* (Sankar and Vidyasagar [11]). Also, *Alg1* generally produces shorter paths than *Bug1* [11]. Hence, *Alg1* may be considered as the desirable choice for any scene.

But, *Alg1* (and *Bug2*) has the following limitation: Under *Alg1*, MA can leave an obstacle boundary only at a point where line segment ST intersects it. This forces MA to travel all the way along the contours of obstacle boundaries, when it could reach T by simply traveling along a straight line. For the example in Fig. 1, this obstacle contour following makes the path P_{Alg1} quite long. For this scene, algorithm *Bug2* also generates the same path.

Also, the task of faithfully following an obstacle boundary can be demanding for the sensing system mounted on MA . The contour following may become even more complex if sensor errors cannot be ignored. If MA follows the obstacle boundary less, and travels more along a straight line, then to some extent the sensor system can be relieved of the contour following task.

The path may be shortened, if MA is allowed to leave an obstacle boundary at any point and move towards target T along a straight line. Then MA does not have to follow an obstacle contour much. A common sense algorithm, say *Com*, due to Lumelsky and Stepanov [7], which achieves this objective is as follows:

Algorithm Com

- (1) Move along a straight line towards T until one of the following occurs:
 - (a) The target T is reached. The procedure stops.
 - (b) An obstacle is met. Follow the obstacle boundary in the prespecified local direction (i.e. left). Go to step 2.
- (2) Leave the obstacle boundary at a point x if MA can move along the line segment xT , towards T . Go to step 1.

The path P_{Com} in Fig. 1 is shorter than P_{Alg1} . Unfortunately, the algorithm *Com* does not always work. Under algorithm *Com*, even in a scene containing only simple obstacles, MA can either meet an infinite number of obstacles (Fig. 2a), or get into an infinite loop (Fig. 3). These limitations were also pointed out by Lumelsky and Stepanov [7].

Since algorithm *Com* has the desirable property of making MA follow the obstacle boundaries less, it is a worth while attempt to rectify the above mentioned flaws of *Com*. A step by step improvement, through the addition of new procedures, transforms the primitive algorithm *Com* into a new algorithm *Alg2*. *Alg2* has robust properties like having the smallest worst case path length possible for a class II algorithm (Sankar [12]), while, at the same time, retaining most of algorithm *Com*'s desirable property.

The process of improving algorithm *Com* also helps in understanding the basic requirements of a class II path planning algorithm. This understanding leads to a search for a possible general underlying principles, which may be common for all the existing class II algorithms. This is elaborated as follows.

The existence of three different class II algorithms, *Bug2*, *Alg1* and *Alg2* naturally raises the following questions on the general characteristics of Class II algorithms

- (1) Though they seem to be based on different ideas, are these three algorithms governed by a common set of some general rules?

- (2) If the answer to the above question is 'yes', then is it possible to generate a family of Class II algorithms, using this set of general rules?
- (3) How is this set of general rules related to problem P1 and the model of scene and MA induced by it? If such a relation is known, it may be possible to modify these rules, whenever some aspects of problem P1 are changed.

This work addresses to all these questions and the presentation is organized as follows: The rules 1, 2 and 3, which are added to Com to form Alg2, are developed in Sections 4 and 5. The procedure for Alg2 is presented in Section 6, followed by comments on the procedure in Section 7. Examples on Alg2's operation are given in Section 8. Section 9 deals with the characteristics of Alg2 like, its worst case path length, local cycle creation, etc. Section 10 presents a general algorithm Gen, of which all the existing class II algorithms are particular cases. Section 11 contains the conclusions and directions for future work.

4. Formulation of Rule 1

The aim of this section is to modify algorithm Com so that MA neither drifts to infinity (Fig. 2a), nor gets trapped in an infinite loop (Fig. 3). This is achieved by imposing some constraints on the leave points defined by the algorithm. A collection of these constraints form rule 1. Rule 1 replaces the step 2 of algorithm Com to form an intermediate algorithm Com1. It is shown that Com1 avoids the situations mentioned earlier. But, Com1 has an undesirable property of producing arbitrarily long paths even for scenes with simple obstacles. Ridding Com1 of this property is the topic of next section.

Consider the situation in Fig. 2a, where MA leaves one obstacle to meet the next in an endless sequence. Then the model assumption that any disc of finite radius intersects only a finite number of obstacles implies that MA drifts to infinity. This can be prevented by imposing additional constraints on the leave points generated by algorithm Com.

Lemma 1: *If step 2 of algorithm Com has an additional restriction that MA leaves an obstacle only in a given disc C, centered at target T and of finite radius R, then MA can meet only a finite number of obstacles R.*

Proof: Let MA, starting from S and moving along line segment ST, meet an obstacle ob1 at a point H. If ob1 does not intersect with disc C, then as per the hypothesis, MA cannot leave obstacle ob1 to meet any other obstacle. That is, MA can meet only one obstacle, and the lemma follows.

Let us assume that obstacle ob1 intersects with disc C. After meeting ob1, MA may or may not leave it. If MA does not leave the only obstacle ob1 it has met, the lemma is proved. On the other hand, if MA leaves ob1 at a point L_1 , then as per the hypothesis L_1 has to lie in C. Similarly, all the leave points L_i ($i \geq 1$) lying on various obstacle boundaries have to be in C. Then, all the line segments L_iT have to be completely in C. Since MA meets an obstacle only when traveling along a line segment L_iT , all obstacles met by MA must intersect with C. Then, from the model assumption that any disc of finite radius intersects only a finite number of obstacles, the lemma follows. ■

Corollary 1: *If step 2 of algorithm Com has a restriction that MA can leave only in disc C, centered at T and of radius D (length of line segment ST), then all the obstacles met by MA must intersect C.*

Hereafter, disc C refers to only that with radius D and center T. Also, algorithm Com, incorporated with the restriction stated in Lemma 1, will be referred to as algorithm Comr.

Leaving an obstacle in disc C only guarantees that, under Comr, MA will not meet an infinite number of obstacles and drift to infinity. But, it does not guard against the possibility of MA

meeting a finite number of obstacles an infinite number of times. That is, MA can get trapped in an infinite loop, while meeting only a finite number of obstacles, as shown in Fig. 3. This type of infinite loop encircles either S or T.

There also exists another type of infinite loop called a self-loop, which does not encircle either S or T. In Fig. 4a, under Comr, MA meets obstacle ob1 at H_1 . After following ob1's boundary MA leaves it at L_1 , to meet ob1 again at an already visited point H_2 . Thus, MA is trapped in an infinite loop containing the points H_2 and L_1 .

The following rule 1, when added to Comr, prevents the occurrence of both types of infinite loops.

Rule 1: *MA leaves an obstacle boundary at a point L, if and only if, both the following conditions are satisfied:*

- (1) *At L, MA can move along line segment LT (i.e. LT does not enter into an obstacle at L).*
- (2) *The Euclidean distance $d(L, T)$ is less than $d(x, T)$, for all x ever visited by MA prior to visiting L (i.e. for all points on the path generated from S up to L).*

Algorithm Com, with its step 2 replaced with rule 1, will be referred to as algorithm Com1. Following examples illustrate the improvement in the performance of Com, when it is modified to Com1.

Unlike the path P_{Com} (Fig. 2a), the path P_{Com1} (Fig. 2b) touches only a finite number of obstacles that intersect with disc C. Unlike the path P_{Comr} (Fig. 4a), the path P_{Com1} (Fig. 4b) does not have any self-loops in it. The issue of infinite loops which enclose either S or T will be considered under general results. Also, Com1 can produce nearly the same path as P_{Com} for the scene in Fig. 1.

A few more words about the infinite loops: Consider the case of T trapped inside an obstacle ob1, and hence being unreachable (Fig. 5). For this scene, the path P_{Com1} , after going around ob1 once, forms an infinite loop which lies entirely on ob1's boundary. It can be shown that, under Com1, an infinite loop which lies entirely on an obstacle boundary occurs, if and only if, either S or T is trapped inside that obstacle. The occurrence of this infinite loop is easily detectable, and also we are mainly interested in cases where T is reachable from S. Hence, for the rest of this work an infinite loop always refers to one which does not lie entirely on an obstacle boundary.

The unreachability condition stated below can also be included as step 3 in algorithm Com1.

If, under Com1, MA returns to a point on an obstacle boundary before leaving that obstacle, then T is unreachable.

These observations are generalized into the following results.

General Results on Algorithm Com1

Lemma 2: *Under algorithm Com1, MA can meet only those obstacles which intersect with the disc C, centered at T and of radius D (length of line segment ST).*

Proof: From condition 2 of rule 1 (step 2 of Com1), it follows that all the leave points L_i generated by Com1 cannot be farther away from T than D. That is, they all have to be disc C. Then, the lemma follows from corollary 1. ■

Lemma 3: *Under algorithm Com1, MA cannot get into any infinite loop.*

Proof: The proof is the same for both types of infinite loops mentioned earlier. Let us assume that MA is trapped in an infinite-loop, under Com1. Then, as discussed earlier, the infinite-loop must contain a portion in which MA travels along a straight line towards T. A part of the infinite loop which contains line segment LH is shown in Fig. 6. That is, MA leaves obstacle ob1 at L

and travels along line segment LT to meet the same or some other obstacle at H . As H is a point on line segment LT other than L , the distance $d(H, T) < d(L, T)$. As H belongs to the infinite-loop, it must have been visited before. That means, L does not qualify for a leave point as an already visited point H is closer to T . This is a contradiction. Hence, the lemma follows. ■

An important question is whether algorithm *Com1* (which does not create any infinite loops) can always produce a finite path length for any scene. Interestingly, the answer is 'no', even for a simple obstacle, as shown below.

Lemma 4: *Algorithm Com1 can produce arbitrarily long path lengths. That is, the worst case path length of Com1 is unbounded.*

Proof: The goal is to construct a scene for which *Com1* produces an arbitrarily long path. Consider a scene (Fig. 7) consisting of a single obstacle $ob1$, with target T being reachable only through a corridor of uniform width W . Let MA first meet $ob1$ at H_1 . After that, the point T is so positioned that MA leaves and meets $ob1$ at points L_1, H_2, L_2, H_3 , etc., all lying in the corridor. That is, MA leaves $ob1$ at L_i , if and only if, $d(L_i, T) < d(H_i, T)$.

From the obstacle construction in Fig. 7, it is clear that as the corridor width W is decreased, the distance between successive leave (or hit) points decreases too. This increases the number of leave (or hit) points. Hence, for any positive integer N , however large, there exists a small enough corridor width W such that, N is the number of leave (or hit) points generated by *Com1*.

Now, let us evaluate the path length after N leave (or hit) points have been generated. The path consists of two parts, namely: The portion lying $ob1$'s boundary and that on line segments L_iH_{i+1} . Let $x > 0$ be the part (e.g. section H_1L_1 in Fig. 7) of obstacle boundary over which MA travels $(N-1)$ times. Then, the path length along $ob1$'s boundary has to be at least $(N-1)x$. From the condition 2 of rule 1 (the closest point condition for a leave point), it follows that the total length of line segments L_iH_{i+1} cannot exceed the length D of line segment ST . That is, the total path length L is mainly decided by the the part $(N-1)x$. Hence,

$$L \geq (N-1)x + D. \quad (4)$$

Since x is of non-zero length, L can be made arbitrarily large, by increasing N (i.e. through decreasing corridor width W). ■

An important implication of the above result is that, though an unbounded path length may be a pathological case, the occurrence of finite but very large path lengths may be quite common under algorithm *Com1*. The large path lengths are due to many repeat trips along already traveled path sections on obstacle boundary. The path lengths can be considerably reduced, if these repeat trips are minimized. Also, it is quite desirable to have a finite and small worst case path length for *Com1*, through some modification.

5. Formulation of Rules 2 and 3

In this section, we make algorithm *Com1* produce only a finite path length for any scene. As pointed out earlier, the repeat trips along sections of obstacle boundary cause arbitrarily long paths. The repeat trips are minimized through the introduction of rules 2 and 3, in algorithm *Com1*. The rule 2 becomes active when MA meets a previously defined hit or leave point. Rule 3 is a constraint on when rule 2 can be applied. The new algorithm *Alg2* is formed by combining algorithm *Com1* and rules 2 and 3. The following treatment mainly motivates the need for rules 2 and 3. The Theorem 1 in Section 9 proves that *Alg2* has a small worst case path length.

Consider the scene in Fig. 8. Under *Com1*, starting from S , MA meets $ob1$ at H_1 . Then it travels along $ob1$'s boundary in the prespecified local direction. After defining a few leave and hit points, MA meets the previously defined hit point H_1 again. Now,

continuing from here with only rule 1 may result in MA visiting H_1 several times (e.g. Fig. 7), leading to a very long path. Hence, a change of strategy in defining MA 's path is called for.

When MA visits H_1 the second time, both the path sections ahead (i.e. section H_1L_1) and behind (section H_3H_1) have already been visited once. At this juncture, MA 's strategy must be to reach an unvisited portion of obstacle boundary, and move along that in search of a leave point. MA can go to either L_1 or H_3 to reach an unvisited section. Let the last defined hit point H_3 be the point to which MA will return (justified in the proof of Theorem 1 in Section 9) along an already traveled path section, say H_1H_3 . After returning to H_3 , MA continues to move along the unvisited portion of obstacle boundary, in search of a new leave point.

This strategy is defined as rule 2.

Rule 2: *If MA meets a previously defined hit or leave point Q after defining a hit point H, then MA returns to the last defined hit point H, via an already traveled path section between Q and H. After returning to H, MA moves along the unvisited section of obstacle boundary, which starts at H.*

Consider again the example in Fig. 8. MA , after returning to the last defined hit point H_3 continues to move along the unvisited section of obstacle boundary starting at H_3 . Then, MA meets the previously defined leave point L_2 . If rule 2 is again applied now, MA must return to the last defined hit point H_3 , and from there proceed to meet the previously defined hit point H_1 . This results in an infinite loop between the points H_1 and L_2 . To prevent this, the following constraint is also needed.

Rule 3: *The rule 2, once applied, can be reapplied only after rule 1 is applied. That is, after MA returns to last defined hit point due to meeting a previously defined hit or leave point (i.e. application of rule 2), the rule 2 can be reapplied only after MA leaves obstacle boundary (i.e. application of rule 1) at least once.*

For the example in Fig. 8, when MA meets previously defined leave point L_2 , the rule 3 prevents rule 2 being applied, as MA has not yet left the obstacle boundary after returning to H_3 .

The new algorithm *Alg2* is formed by combining algorithm *Com1* and rules 2 and 3. A higher level description of *Alg2* is as follows:

A higher level description of Alg2

- (1) MA travels along a straight line towards T until an obstacle is met.
- (2) Rule 1 decides the point at which it can leave that obstacle.
- (3) If a previously defined hit or a leave point is met, rule 2 is applied.
- (4) Rule 2 is reapplied only after rule 1 is applied at least once.
- (5) If T is not reachable, that can be detected by checking some conditions

A detailed procedure of *Alg2* based on this higher level description will be presented in the next section.

The application of rules 2 and 3 results in an efficient way of searching an obstacle boundary for a leave point. That is, under algorithm *Alg2*, MA does not visit any point on an obstacle boundary more than "twice". This can be compared with the fact (Lemma 4) that, under *Com1*, MA may visit a point on obstacle boundary any number of times. That means, under *Alg2* the path length along obstacle boundaries cannot be more than twice the sum of perimeters of obstacles met by MA . The characteristics of *Alg2* are studied in Section 9.

Alg2, like *Alg1* (Sankar and Vidyasagar [11]), incorporates an optimization procedure to minimize MA 's trips along already traveled path sections. This helps reduce the overall path length. The

example in Fig. 8 illustrates this procedure. Here, *MA* can return to the last defined hit point H_3 from hit point H_1 , in two ways. It can either reverse its direction of motion and retrace the path section H_3H_1 , or reach H_3 via the already created path section $H_1L_1H_2L_2H_3$. The algorithm *Alg2* chooses the shorter of these two path sections to return to H_3 , which in this case is H_3H_1 .

Next, we present the procedure for *Alg2*, followed by comments on various steps.

6. The Procedure for Alg2

The algorithm can be followed using the example shown in Fig. 8. The goal is to generate a continuous path from S to T . The algorithm is to be executed at any point on the path. The hit points and leave points are arranged in a sequence in the order of their occurrences. The subscript j denotes the j th occurrence of a hit or a leave point.

The following two pieces of information are stored along with each hit point H_j :

- (1) $d_p(S, H_j)$, which is the distance along the path (i.e. length of the path) from S up to hit point H_j .
- (2) The local direction at H_j , which is initially set as *left* for all the hit points. This could be reset to *right* under certain conditions as described in the algorithm. If *MA* ever meets H_j again, when traveling along a line segment towards T , then it follows the obstacle boundary at H_j in the local direction stored with H_j .

Along with each leave point L_j , the distance $d_p(S, L_j)$ is stored. From the ordering of hit and leave points, it is possible to identify the leave point L_j , which occurs immediately after a hit point H_j .

To determine the unreachability of target T , a certain leave point is stored in register U .

A variable Q is initialized to a value D . At every point x on the path, the Euclidean distance $d(x, T)$ is computed and compared with the value Q . If $d(x, T)$ is less than Q , then Q is made equal to $d(x, T)$. That is, Q is the shortest distance between target T and any point on the path generated from S up to *MA*'s current position.

The path length from S up to *MA*'s current position is computed and stored in the path length counter PC .

The algorithm consists of the following steps:

- (0) Set $j = 1$, $L_{j-1} = S$, $Q = D$, $PC = 0$.
- (1) From L_{j-1} , move along line segment $L_{j-1}T$ until one of the following occurs:
 - (1a) Target T is reached. The procedure stops.
 - (1b) An obstacle is met. Define a hit point H_j . Store $d_p(S, H_j)$ and local direction *left* along with H_j . Turn left and follow obstacle boundary. Go to 2.
- (2) From hit point H_j , move along obstacle boundary until one of the following occurs:
 - (2a) Target T is reached. The procedure stops.
 - (2b) A point P is reached such that $d(P, T)$ is less than Q and *MA* is free to move along the line segment PT . Define a leave point L_j at P and store the distance $d_p(S, L_j)$ along with it. Set $j = j + 1$. Go to 1.
 - (2c) A previously defined hit or leave point Q_k , ($k < j$) is met. Let d_c be the current content of path counter PC (i.e. the path length up to this point). The following steps are executed:
 - (2c1) If $(d_1 \geq d_2)$

MA's direction of motion is reversed and *MA* retraces its path back to H_j . Once H_j is reached, start incrementing PC . Go to 2c3.
 - (2c2) If $(d_1 < d_2)$,

MA moves along the same path section already generated between the points Q_k (i.e. H_k or L_k) and H_j , when *MA* passed through Q_k the first time. That is, *MA* will pass through all the previously defined hit and the leave points between Q_k and H_j .

MA moves along obstacle boundary from Q_k (without reversing its direction). If a previously defined leave point L_i , ($k \leq i < j$) is met, *MA* leaves the obstacle boundary and moves along line segment L_iT . This takes it to hit point H_{i+1} . At H_{i+1} , *MA* checks the local direction stored with that hit point and moves along obstacle boundary accordingly. This procedure of retracing the previously defined hit and the leave points takes *MA* back to H_j . At H_j , *MA* moves along obstacle boundary in the associated local direction (i.e. right). The path counter PC is started again. Go to 2c3.
 - (2c3) From H_j , continue moving along obstacle boundary until one of the following occurs.
 - (a) Target T is reached. The procedure stops.
 - (b) A point P is met, such that, $d(P, T)$ is less than Q and *MA* is free to move along line segment PT . Define a leave point L_j at P and store the content of PC (i.e. $d_p(S, L_j)$) along with it. Set $j = j + 1$. Go to 1.
 - (c) *MA* reaches a leave point stored in U . Target T is not reachable as either S or T is trapped inside an obstacle. The procedure stops.
- (2d) *MA* returns to H_j without defining any new leave point. Again, target T is not reachable. The procedure stops.

The local direction at H_j is reset to *right*.

If Q_k is a hit point H_k , then the leave point L_k is stored in U .

The quantities d_1 and d_2 are evaluated as follows. d_1 and d_2 are respective lengths of the two path sections along either one of which *MA* can return to H_j from Q_k .

$$d_1 = d_p(S, H_j) - d_p(S, Q_k) \quad (5)$$

$$d_2 = d_c - d_p(S, H_j) \quad (6)$$

The quantities $d_p(S, H_j)$ and $d_p(S, Q_k)$ are available as stored along with the points H_j and Q_k , respectively.

Reset the content of path counter PC to $d_p(S, H_j)$. Stop incrementing PC until *MA* returns to hit point H_j .

Next, the distances d_1 and d_2 are compared to decide the return path for *MA*, from Q_k to H_j .

- (2c1) If $(d_1 \geq d_2)$

MA's direction of motion is reversed and *MA* retraces its path back to H_j . Once H_j is reached, start incrementing PC . Go to 2c3.

- (2c2) If $(d_1 < d_2)$,

MA moves along the same path section already generated between the points Q_k (i.e. H_k or L_k) and H_j , when *MA* passed through Q_k the first time. That is, *MA* will pass through all the previously defined hit and the leave points between Q_k and H_j .

MA moves along obstacle boundary from Q_k (without reversing its direction). If a previously defined leave point L_i , ($k \leq i < j$) is met, *MA* leaves the obstacle boundary and moves along line segment L_iT . This takes it to hit point H_{i+1} . At H_{i+1} , *MA* checks the local direction stored with that hit point and moves along obstacle boundary accordingly. This procedure of retracing the previously defined hit and the leave points takes *MA* back to H_j . At H_j , *MA* moves along obstacle boundary in the associated local direction (i.e. right). The path counter PC is started again. Go to 2c3.

- (2c3) From H_j , continue moving along obstacle boundary until one of the following occurs.

- (a) Target T is reached. The procedure stops.

- (b) A point P is met, such that, $d(P, T)$ is less than Q and *MA* is free to move along line segment PT . Define a leave point L_j at P and store the content of PC (i.e. $d_p(S, L_j)$) along with it. Set $j = j + 1$. Go to 1.

- (c) *MA* reaches a leave point stored in U . Target T is not reachable as either S or T is trapped inside an obstacle. The procedure stops.

- (2d) *MA* returns to H_j without defining any new leave point. Again, target T is not reachable. The procedure stops.

7. Comments on the Steps of Alg2

- (1) Step 2b implements rule 1 to determine the leave points.
- (2) The step 2c implements the rule 2, which dictates that *MA* must return to the last defined hit point, if it encounters a previously defined hit or a leave point.

The quantities d_1 and d_2 are the respective lengths of the two already generated path sections either of which can take MA from Q_k (a previously defined hit or leave point) to the last defined hit point H_j . The algorithm picks the shorter of these two paths by comparing d_1 and d_2 . Steps 2c1 and 2c2 generate, respectively, each of these return paths.

- (3) The rule 3 is implemented by having step 2b (rule 1) repeated inside step 2c (rule 2), as step 2c3. This is done to prevent the reapplication of rule 2, before rule 1 is applied.
- (4) In step 2c, when a previously defined hit or leave point is met, the path counter PC is reset to the path length stored with the last defined hit point (i.e. $d_p(S, H_j)$). Also, the incrementing of the path counter is started only after MA returns to H_j . Due to these procedures, when MA returns to H_j , the trip it made between H_j and Q_k is not taken into the overall path length. Also, if MA meets H_j again when moving along a line segment towards T , the local direction *right* stored with H_j ensures that MA won't repeat the trip between H_j and Q_k .
- (4) Terminating condition 2d is the same as in *Bug2*. Another new terminating condition 2c3.c is also needed for *Alg2*. The rationale behind this terminating condition will be discussed in Section 9.

8. Examples

The Fig. 9 shows a scene with the path generated by *Alg2*. For the same scene, the path generated by *Alg1* is shown in Fig. 10. As the target T is located close to a wall, under *Alg2*, MA does not leave the obstacle until it comes very close to T . This results in a shorter path as compared to P_{Alg1} (Fig. 10) which contains local cycles. But, we can also easily find scenes in which *Alg2* produces longer paths with local cycles, when *Alg1* does not produce local cycles. Hence, it is generally not possible to compare the path lengths produced by *Alg1* and *Alg2*. Fig. 11 shows the path generated by algorithm *Bug2* for the scene in Fig. 9. The path P_{Bug2} is much longer compared to P_{Alg1} and P_{Alg2} , because MA makes three repeat trips along most of the obstacle boundary.

9. Characteristics of Alg2

We are interested in finding the worst case path length of *Alg2*.

Theorem 1: Under algorithm *Alg2*, for any scene, the path length L between the point S and T , cannot exceed the limit,

$$M = 2 \sum p_i + D, \quad (7)$$

where p_i is the perimeter of the i_{th} obstacle met by MA , and D is the length of line segment ST .

Proof: The proof mainly highlights the essential ideas, and some intermediate results are not fully proved, for space considerations. Please refer to Sankar [12] for a complete proof.

Let us consider a single obstacle $ob1$. The obstacle boundary, which is a simple closed curve, can be represented in a simple way (i.e. through a continuous deformation to more convex-like), for easy visualization (Fig. 12). The straight line segments in the path are shown as curved lines due to the above mentioned continuous deformation of the scene. An interval ab along this obstacle boundary is defined as the section of the boundary between the points a and b , when going in the clock-wise direction from a to b .

Under algorithm *Alg2*, starting from S , MA first meets $ob1$ at H_1 . Then it moves along $ob1$'s boundary until a leave point L_1 is defined. Moving along line segment L_1T , MA meets $ob1$ again at H_2 and so on, to form a non-crossing and non-overlapping pattern as shown in Fig. 12. The condition 2 of rule 1 restricts a leave point L_i to be the closest to T than any other point on the path generated from S up to L_i . It can be easily shown that this condition prevents the path P_{Alg2} from crossing or overlapping with itself.

Assume that no local cycles are present in the path. Then the non-overlapping pattern of hit and leave points in Fig 12 implies that, before returning to the first defined hit point H_1 , MA reaches T while traveling along a line segment L_mT , where L_m is the m_{th} leave point. As MA has traveled only once along any interval of the path, the path length along obstacle boundaries cannot exceed $\sum p_i$. The total length of the path sections along line segments L_iH_{i+1} , ($i \leq m$) cannot exceed the distance D between S and T . This again follows from condition 2 of rule 1. Hence, the Theorem holds for this case.

Now, let us assume that the path contains local cycles. Then, the non-overlapping pattern in Fig. 12 again implies that, after defining a hit point H_n and moving along obstacle boundary, MA must meet the previously defined hit point H_1 again. Here, n can be any finite positive integer.

The intervals L_iH_{i+1} , ($i = 1, n-1$) along obstacle boundary have not been visited by MA yet, as MA moved along line segments L_iH_{i+1} , instead. The intervals H_iL_i , ($i = 1, n-1$) and H_nH_1 have been visited by MA once.

MA , having returned to the first defined hit point H_1 , must proceed to search the unvisited intervals L_iH_{i+1} along the obstacle boundary, to find a leave point L^T to reach T . That is, line segment $L^T T$ will not meet the obstacle again. But, every unvisited interval is sandwiched between two once-visited intervals. This makes it necessary for MA to repeat its trip at least one more time along the once-visited intervals.

It is quite important to devise an efficient strategy to search the unvisited intervals, so that a search can be conducted with minimum possible repeat trips along already visited path sections. It will be shown next, that the rules 2 and 3 of *Alg2* indeed form such a strategy which ensures that MA does not visit any point on the obstacle boundary more than "twice".

An efficient way to search the unvisited intervals is as follows: From H_1 , move in the counter clock-wise direction (along the once-visited interval H_nH_1 on the obstacle boundary) to enter the unvisited interval $L_{n-1}H_n$. Again, the non-crossing of paths due to condition 2 of rule 1, ensures that if MA leaves obstacle boundary within the unvisited interval, it must meet the obstacle (if it has not reached T) again within the same interval. Having entered the unvisited interval $L_{n-1}H_n$, MA must leave it only after completely searching it (i.e. not leaving behind any unvisited subinterval within $L_{n-1}H_n$).

The following result is stated without proof, for space considerations. The proof, though straightforward, is quite detailed (Please refer Sankar [12]).

It is always possible to search an unvisited interval completely (i.e. without leaving any subinterval unvisited) before exiting it. Also, during the search MA will not meet any point in the unvisited interval more than "twice". The application of rules 1,2 and 3 makes this possible.

After searching interval $L_{n-1}H_n$ completely, MA moves to the next unvisited interval in the counter clock-wise direction (i.e. interval $L_{n-2}H_{n-1}$) through the once-visited interval $H_{n-1}L_{n-1}$. This process is repeated until all the unvisited intervals are searched or target T is reached. It is quite clear that MA , after completely searching an unvisited interval and exiting it, never returns to that interval again. Also, to go from one unvisited interval to the next, MA moves along an once-visited interval, only one more time.

From these observations it follows that, MA does not visit any point on the obstacle boundary more than twice. That means, the path length along the obstacle boundary cannot exceed $2p$, where p is the perimeter of the obstacle. Since all the repeat paths are on obstacle boundary, MA moves along the line segments L_iH_{i+1} only once. As pointed out in the 'no local cycle' case, the total length of these line segments cannot exceed D .

Hence the total path length L cannot exceed the limit $2p + D$.

It can be shown, along the same lines, that for a multiple obstacle case, the path length L cannot exceed the limit $2 \sum p_i + D$, where p_i is as defined in the Theorem. ■

Comments On Theorem 1:

- (1) The number n of hit or leave points can be very large, though finite. But, this does not alter the worst case path length of *Alg2* as it is independent of the scene. For algorithm *Bug2*, the worst case path length (Eqn. 2) is quite dependent on the scene.
- (2) The procedure *Alg2* requires storing of each hit and leave point and the path length information along with it. The memory requirement for this can be reduced by erasing a part of this information as *MA* moves on. For example, once an unvisited interval is completely searched and exited the information on all the hit and leave points in that interval can be erased as *MA* will never return to it.
- (3) The unreachability condition 2c3.c also follows from the proof. For example, consider *MA* returning to hit point H_1 . The section H_1L_2 along obstacle boundary is once-visited. When searching the unvisited intervals in the counter clock-wise direction, if *MA* meets L_2 again, it only means that *MA* has covered the whole boundary and couldn't find a leave point. This idea can be generalized to the following condition:

Unreachability Condition: Let *MA* meet a previously defined hit point H_j , after defining a hit point H_k , ($k > j$). After returning to last defined hit point H_k , if *MA* meets the leave point L_j which was defined after H_j , then target T is not reachable.

- (4) It can be shown that the upper bound $2 \sum p_i + D$ on the path length (worst case path length) is indeed a tight bound. That is, it is possible to construct a scene such that the path length generated is as close to the upper bound as desired.
- (5) The searching strategy of *Alg2* is optimal in the sense that the worst case path length cannot be reduced any further, as it is the minimum possible for any class II algorithm [12].

9.2 The optimization Procedure

The optimization procedure in *Alg2* chooses the shorter of two possible path segments from a previously defined hit or leave point Q to the last defined hit point H . When the local cycle are created with multiple obstacles, then there exist more than two path sections to go from Q to H . It can be shown that up to $n+1$ such path sections are available between Q and H for an n obstacle case. An efficient data structure can store all these possible return paths and select the shortest among them. Provided with this feature, *Alg2* can produce shorter paths than in its present form.

9.3 The Local Cycles

The local cycle creation under *Alg2* is quite different from that under *Alg1* (or *Bug2*). For example, under *Alg2* local cycles are possible even when all the obstacles are convex. On the other hand, due to the positioning of T a local cycle may not be created in a complex scene (Fig. 9). The local cycle is not really a serious issue for both *Alg1* and *Alg2* as the number of visits along any section of obstacle boundary is limited to two. On the other hand, when local cycles are created under *Bug2* *MA* can visit a section of an obstacle boundary up to $n/2$ times, where n is the number of intersections line segment ST makes with that obstacle. As the number n becomes larger the resulting path length can be very long.

10. A General Algorithm Gen

The step by step development of algorithm *Alg2* from the primitive algorithm *Com* has brought out the essential requirements of a class II algorithm. Rule 1 prevents *MA* from drifting to infinity or getting into infinite loops. Rule 2 ensures a finite

path length for any given scene. Both these rules are essential for algorithm *Alg2*'s operation. For example, if rule 1 is not present, then rule 2 alone cannot prevent *MA* from drifting to infinity or getting trapped in an infinite loop. Similarly, without rule 2 finite path lengths cannot be guaranteed.

Interestingly, rule 1 has a scope for generalization. The closest point condition of rule 1 (i.e. condition 2 of rule 1), for defining a leave point, uses the Euclidean distance in the case of algorithm *Alg2*. The choice of any other metric must be equally valid. Hence, the rule 1 can be redefined in a more general fashion as follows:

Let T be the origin of the coordinate system to denote the points in the scene. For convenience, let a rectangular coordinate frame with its x -axis along the line segment ST be chosen. Then, for any point X in the plane, the distance $d(X, T)$ is the norm of that point. The norm of a point X is given by,

$$\|X\| = ((w_1x)^p + (w_2y)^p)^{1/p}, \quad (8)$$

where (x, y) are the coordinates of X in the chosen frame, the weights w_1 and w_2 are some real numbers, and p is any positive integer including $+\infty$.

In this coordinate frame the generalized rule 1 can be defined as follows:

Generalized Rule 1: *MA* leaves an obstacle boundary at a point L , if and only if, both the following conditions are satisfied:

- (1) At L , *MA* can move along line segment LT (i.e. LT does not enter into an obstacle at L).
- (2) The $\|L\|$ (i.e. $d(L, T)$) is less than $\|X\|$ (i.e. $d(X, T)$), for all X ever visited by *MA* prior to visiting L (i.e. for all points on the path generated from S up to L).

The higher level description of the generalized algorithm *Gen* is identical to that of *Alg2* (Section 5), except that generalized rule 1 replaces rule 1. That is, the rules 2 and 3 and the terminating conditions remain the same as in *Alg2*.

By choosing different norms (i.e. different p and different weights w_1 and w_2) in the generalized rule 1, several variations of the general algorithm *Gen* can be generated.

Interestingly, the worst case path length is the same (i.e. $2 \sum p_i + D$, where the terms are as defined before) for all these variations of algorithm *Gen*. This is evident from the fact that the proof on the worst case path length of *Alg2* (Theorem 1) holds good even when any distance measure other than the Euclidean is used.

Algorithm *Alg1* is obtained from *Gen* by setting $p = 1$, $w_1 = 1$, $w_2 = \infty$. Hence, for any point $X = (x, y)$ in the scene, the $\|X\|$ is unbounded everywhere, except when X lies on the x -axis, which is the line ST . Also, the norm (i.e. $d(X, T)$) of any point X on the line ST is merely the absolute value of its x -coordinate. From these, it follows that starting from S , *MA*, can leave an obstacle only on the line ST , and also, if and only if, X is the closest point to T on line ST , ever visited by *MA*. This is exactly the version of rule 1 for algorithm *Alg1* [11]. Hence, with the above choice of norm the general algorithm *Gen* becomes algorithm *Alg1*.

Similarly, the algorithm *Gen* becomes algorithm *Alg2*, when unweighted Euclidean norm is chosen. That is, $p = 2$, $w_1 = 1$, $w_2 = 1$.

The algorithm *Bug2* (Lumelsky [6]), under which *MA* leaves only on line segment ST is nothing but algorithm *Alg1* without rules 2 and 3. That is, *Bug2* is formed only out of rule 1 (described above) of *Alg1*. Hence, *Bug2* is to *Alg1*, what algorithm *Com1* (Section 4) is to *Alg2*. It can be recollected that Lemma 4 shows that *Com1* can produce arbitrarily long paths. The same is true for *Bug2* too. The worst case path length of *Bug2* is $\sum p_i n_i / 2 + D$, where n_i is the number of intersections the line segment ST makes with an obstacle of perimeter p_i . The worst case

path length can be arbitrarily large as the number n_i , though finite, can be arbitrarily large. Nevertheless, there is a difference between *Bug2* and *Com1*. A simple obstacle (Fig. 7), which does not intersect any straight line at more than four points, can cause a long path under *Com1*. On the other hand, to cause a long path under *Bug2*, an obstacle has to be quite complex, intersecting line segment *ST* at many points (Fig. 11). This probably is why this aspect of *Bug2* has not been noticed before. It must be mentioned here that, in this respect, *Alg2* and *Alg1* are distinct from their respective prototypes *Com1* and *Bug2*. That is, for both *Alg1* and *Alg2*, the worst case path length is a constant, which does not upon the scene at all.

Thus a generalization of the existing class II algorithms has given a good insight into their operating principles.

11. Conclusions

A new algorithm *Alg2* was developed by adding new procedures to a primitive algorithm *Com*. Unlike other existing algorithms, under *Alg2*, *MA* can leave an obstacle anywhere along its boundary. This results in *MA* traveling less along obstacle boundaries and more along line segments towards target *T*. Also, the worst case path length of *Alg2* is the minimum possible for any class II algorithm. The development process of *Alg2* has resulted in the creation of a general algorithm of which the three existing algorithms *Bug2*, *Alg1* and *Alg2* are particular variations. The generalization also led to a better understanding of the operating principles of a path planning algorithm.

The fact that *Alg2* does not use the line segment *ST* for deciding the leave points can be used to generalize the problem *P1*. For example, the target *T* can be a connected region or a collection of connected regions, instead of just being a point. Only the rule 1 of *Alg2* needs to be modified to handle this. That is, $d(L, T)$ will then refer to the distance between the point *L* and the set *T*. The non-uniqueness of the direction to move towards *T* can be resolved based on some other considerations.

This can be useful in manipulator obstacle avoidance, where an end-effector goal position corresponds to multiple target points in joint-space. Also, a point *T* can be modeled as a region to account for sensor error. The effect of sensor error on the performance of these algorithms is being studied.

12. References

- [1] *Algorithmic Motion Planning*, C. K. Yap, Advances in Robotics, Vol. 1: Algorithmic and Geometric Aspects (J. T. Schwartz and C. K. Yap Eds.), 1987.
- [2] *Real-Time Obstacle Avoidance for Manipulators and Mobile Robots*, O. Khatib, The International Journal of Robotics Research, Vol. 5, No. 1, 1986.
- [3] *Obstacle Avoidance for Kinematically Redundant Manipulators in Dynamically Varying Environments*, A. A. Maciejewski and C. A. Klein, The International Journal of Robotics Research, Vol. 4, No. 3, 1985.
- [4] *Some Heuristics for the Navigation of a Robot*, R. Chattergy, The International Journal of Robotics Research, Vol. 4, No. 1, 1985.
- [5] *Turtle Geometry*, H. Abelson and A. diSessa, MIT Press, Cambridge, MA, 1980, pp. 176-199.
- [6] *Path Planning Strategies For A Point Mobile Automaton Moving Amidst Unknown Obstacles Of Arbitrary Shape*, V. J. Lumelsky, A. A. Stepanov, Algorithmica, Springer-Verlag, 1987.
- [7] *Dynamic Path Planning For A Mobile Automaton With Limited Information On The Environment*, V. J. Lumelsky, A. A. Stepanov, IEEE Transactions On Automatic Control, Vol. AC-31, No. 11, Nov. 1986.
- [8] *Dynamic Path Planning For A Planar Articulated Robot Arm Moving Amidst Unknown Obstacles*, V. J. Lumelsky, Automatica, Journal Of International Federation Of Automatic Control, June 1987.
- [9] *Effect of Kinematics On Dynamic Path Planning For Planar Arms Moving Amidst Unknown Obstacles*, V. J. Lumelsky, IEEE Journal of Robotics And Automation, May, 1987.
- [10] *Continuous Motion Planning In Unknown Environment For A 3D Cartesian Robot Arm*, V. J. Lumelsky, IEEE Conference On Robotics And Automation, 1986.
- [11] *A New Path Planning Algorithm For A point Object Moving Amidst Unknown Obstacles In A Plane*, A. Sankaranarayanan And M. Vidyasagar, IEEE Conference On Robotics And Automation, 1990.
- [12] *Path Planning For Moving A Point Object Amidst Unknown Obstacles In A Plane: New Algorithms, Universal Bounds And Generalizations* A. Sankaranarayanan, Ph. D. Thesis, University Of Waterloo, Waterloo, Canada, 1990.

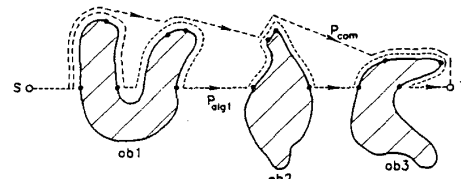


Fig. 1 A scene with the paths generated by algorithms *Alg1* and *Com*

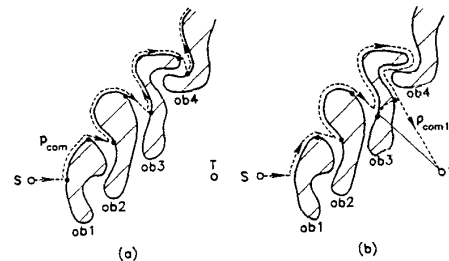


Fig. 2 *MA* drifts to infinity under algorithm *Com*, but meets only a finite number of obstacles under algorithm *Com1*

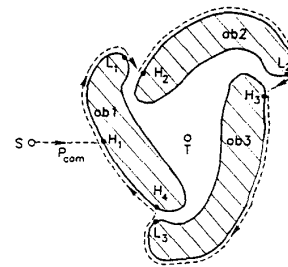


Fig. 3 *MA* is trapped in an infinite loop under algorithm *Com*

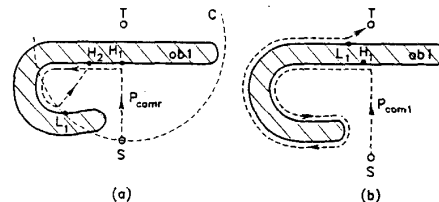


Fig. 4 Algorithm *Comr* produces a self-loop, while algorithm *Com1* does not, for the same scene.

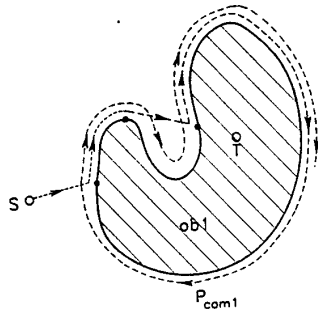


Fig. 5 The infinite loop under *Com1* lies entirely on *Ob1*'s boundary

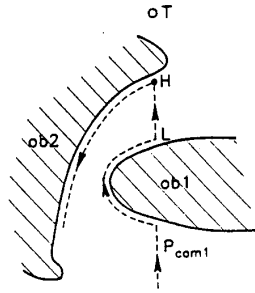


Fig. 6 A part of an infinite loop under algorithm *Com1*

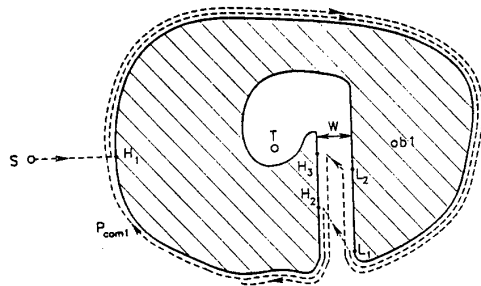


Fig. 7 A scene in which algorithm *Com1* can produce arbitrarily long paths

